# Problem Set 0 - Getting Started

## 1 Task 1: Installation

Go through the following installation and getting started steps:

1. **Installation:** Follow the intallation guide for *Anaconda* and *VSCode*.
   Also sign up for *Github Copilot Pro*.
2. **VSCode+Jupyter:** Create a *Jupyter Notebook* in *VSCode*.
   Copy in the code below and run it.

```
a = 1
b = 2
c = a + b
print(c)
```

3. **More VSCode:** Try out the tips for *VSCode* found here.
   (creating the *Restart-and-Run-All* short cut will be very useful)

*The following tasks is designed to be done after Lecture 3, but you can experiment with it before. Task 2 and 4 are repeated on Problem_set_0.ipynb.*

## 2 Task 2: Understanding code

Consider the code snippets below. For each, write down your expected outcome on paper. Run the code and check whether you were correct.

- **indexing and references**

```
x = [1,2,3]
y = x
y[-1] = 4
print(x)
```

- **slicing**

```
x = [0,1,2,3,4,5]
print(x[:2])
print(x[2:])
```

- **loops - break**

```
for i in range(5):
    if i >= 2: break
    print(i)
```

- **loops - continue**

```
for i in range(5):
    if i == 2: continue
    print(i)
```

- **conditionals**

```
x = 3
if x > 3:
    print('too big')
elif x < 1:
    print('too small')
else:
    print('just right')

```

- **functions and scope**

```
a = 1
def f(x):
    return x+a # a is global variable
def g(x,a=1):
    return x+a # a is local variable
print(f(1))
print(g(1))
a = 2
print(f(1))
print(g(1))
```

- **dictionaries**

```
d = {'a': 1, 'b': 2}
for key in d:
    d[key] = d[key]*2
print(d['a'] + d['b'])
print('c' in d)
```

- **list comprehension and zip**

```
a = [1,2,3]
b = [4,5,6]
c = [i+j for i,j in zip(a,b)]
print(c)
```

- **floating point numbers**

```
print(0.1 + 0.2 == 0.3)
import math
print(math.isclose(0.1 + 0.2, 0.3))
```

- **classes**

```
class SquareClass:
    def __init__(self,length,width):
        self.length = length
        self.width = width
    def size(self):
        return self.length*self.width
square = SquareClass(2,2)
print(square.size())
```

- **numpy - view and copies I**

```
import numpy as np
x = np.array ([1,2,3])
y = x
x += 1
print(y)
x[:] = x + 1
print(y)
x = x + 1
print(y)
```

- **numpy - view and copies II**

```
import numpy as np
A = np.array([0,1,2,3,4])
```

```
B = A[1:4]
B[:] = 0
print(A)
```

- **numpy - broadcasting**

```
import numpy as np
A = np.array([[1,2,3],[4,5,6]])
b = np.array([10,20,30])
print(A + b)
```

## 3 Task 3: Try Copilot

Ensure c

1. **Automatic:** Create empty notebook. Type in the code below and wait for the copilot to suggest the next lines of code.

   Accept: Tab

   Reject: Esc

```
import numpy as np
x = np.array([2,5,10,2,4])
# sort and print
```

2. **Chat:** You can toggle the right hand side chat window by pressing Ctrl+Alt+B. Try this and then write in the chat: "plot x vs log(x)".
3. **Snooze copilot:** Hover over the Copilot icon to the right in the Status Bar and pres `Snooze`.

## 4 Task 4: Putting it together

1. **Grid search (numpy):** Create a numpy array `x` of 100 evenly spaced points between 0.01 and 10 (use `np.linspace`). Let utility be $u(x) = \sqrt{x}$. Without writing a loop, find and print the value of `x` for which $u(x)$ is closest to 2. Hint: combine `np.abs` and `np.argmin`.
2. **Broadcasting (numpy):** Let `p = np.array([1,2,5])` be prices and `q = np.array([10,20,30,40])` be quantities. Without any loop, construct the $4 \times 3$ matrix `E` where `E[i,j] = q[i]*p[j]`. Print `E` and `E.shape`. Hint: use `None` to add a new axis, e.g. `q[:,None]`.
3. **A class:** Write a class `ConsumerClass` that stores income `I` and prices `p1`, `p2` as attributes (set in`__init__` with sensible defaults that can be overridden). Add a method `demand(alpha)` that returns the Cobb-Douglas demands $(x_1, x_2) = (\alpha I/p_1, (1-\alpha)I/p_2)$ as a tuple, and a`__str__` method that returns a readable summary. Create two consumers with different incomes and print their demands for $\alpha = 0.5$.